

Implementação do Compilador C-



Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Discente: José Augusto da Costa Caldeira

Universidade Federal de São Paulo- UNIFESP Instituto de Ciência e Tecnologia-
Campus São José dos Campos

São José dos Campos- Brasil

Julho de 2025

1. Introdução

Os compiladores representam uma das tecnologias mais fundamentais da ciência da computação, atuando como a ponte indispensável entre a cognição humana e a lógica binária das máquinas. Eles são os tradutores sofisticados que convertem o código-fonte, escrito em linguagens de alto nível e próximo da nossa forma de pensar, para a linguagem de máquina, a única que um processador pode executar diretamente. Sem os compiladores, a programação moderna seria uma tarefa hercúlea, restrita a um pequeno grupo de especialistas capazes de manipular instruções de baixo nível, um processo não apenas tedioso e complexo, mas também específico para cada tipo de hardware existente.

O papel de um compilador, no entanto, transcende a mera tradução. Ele eleva o nível de abstração, permitindo que desenvolvedores se concentrem na resolução de problemas e na lógica de seus algoritmos, em vez de se perderem nos detalhes intrincados da arquitetura do processador. Além disso, um compilador robusto é um pilar para a qualidade do software: ele realiza a detecção precoce de erros, aplica otimizações que tornam os programas mais rápidos e eficientes, e garante a portabilidade do código entre diferentes sistemas, um conceito essencial no mundo tecnológico atual.

Este projeto documenta a concepção e implementação de um compilador para a linguagem C- (uma versão simplificada da linguagem C), com o objetivo de gerar código para um processador de arquitetura RISC, baseado no histórico e influente padrão MIPS, desenvolvido previamente. Este trabalho prático permitiu solidificar os conceitos teóricos da disciplina, conectando de forma tangível a engenharia de software (o compilador) com a engenharia de hardware (o processador para o qual foi projetado).

2. O Processador-Alvo: Arquitetura e História

A eficácia de um compilador está intrinsecamente ligada à arquitetura do processador para o qual ele gera código. O processador-alvo deste projeto foi desenhado com base em uma das arquiteturas mais icônicas da história da computação: a MIPS.

2.1. A Filosofia RISC e a História da MIPS

A arquitetura MIPS (acrônimo para *Microprocessor without Interlocked Pipeline Stages*) foi desenvolvida originalmente por uma equipe liderada por John L. Hennessy na Universidade de Stanford, em 1981, e se tornou um dos exemplos mais puros e bem-sucedidos da filosofia **RISC (Reduced Instruction Set Computer)**. O conceito RISC surgiu como uma resposta à crescente complexidade das arquiteturas

CISC (Complex Instruction Set Computer), que dominavam o mercado. A aposta da filosofia RISC era que um conjunto menor e mais simples de instruções, executadas em um único ciclo de clock e com formatos de tamanho fixo, permitiria a construção de processadores mais rápidos, eficientes e com um design mais limpo.

O processador desenvolvido para este projeto adere a esses princípios. Ele implementa um modelo de ciclo único (monociclo), em que cada instrução de 32 bits é executada em um único ciclo de clock. Adicionalmente, adota uma **arquitetura Harvard**, que se caracteriza

por possuir barramentos e memórias separadas para dados e instruções, uma abordagem que permite buscas simultâneas e melhora o desempenho ao reduzir gargalos de acesso à memória.

2.2. Diagrama de Blocos e Componentes do Processador

O diagrama de blocos abaixo ilustra a estrutura geral do processador, destacando seus principais componentes funcionais e suas interconexões.

(Figura 1: Diagrama de blocos do Processador)

A seguir, uma descrição detalhada dos componentes:

- **Contador de Programa (PC):** Registrador de 32 bits que armazena o endereço da próxima instrução a ser buscada e executada.
- **Memória de Instruções:** Uma memória ROM de 64 posições de 32 bits, responsável por armazenar o código de máquina gerado pelo compilador.
- **Banco de Registradores:** Uma memória de trabalho de alta velocidade, composta por 64 registradores de 32 bits, que servem como operandos para a ULA.
- **Unidade de Controle:** O cérebro do processador, que decodifica o *opcode* de cada instrução e gera os sinais de controle que orquestram a operação de todos os outros componentes.
- **Unidade Lógica e Aritmética (ULA):** Realiza todas as operações de cálculo (aritméticas e lógicas) do processador.
- **Memória de Dados:** Uma memória RAM de 64 posições de 32 bits, utilizada para armazenar e acessar variáveis durante a execução do programa.

2.3. Conjunto de Instruções e Organização da Memória

O processador executa um conjunto de instruções de 32 bits que se enquadram em categorias como:

INSTRUÇÃO	OPERAÇÃO	TIPO
Operações aritméticas		
Add [RD, RS, RT]	$RD \leftarrow RS + RT$	R
Sub [RD, RS, RT]	$RD \leftarrow RS - RT$	R
Div [RD, RS, RT]	$RD \leftarrow RS / RT$	R
Mult [RD, RS, RT]	$RD \leftarrow RS * RT$	R
AddI [RD, RS, IM14]	$RD \leftarrow RS + IM14$	I
SubI [RD, RS, IM14]	$RD \leftarrow RS - IM14$	I
DivI [RD, RS, IM14]	$RD \leftarrow RS / IM14$	I
MultI [RD, RS, IM14]	$RD \leftarrow RS * IM14$	I
Operações lógicas		
Or [RD, RS, RT]	$RD \leftarrow RS RT$	R

And [RD, RS, RT]	RD <= RS & RT	R
Xor [RD, RS, RT]	RD <= RS ^ RT	R
Not [RD, RS]	RD <= ~RS	R
Sgt	RD <= RS > RT ? 1 : 0	
Slt	RD <= RS < RT ? 1 : 0	
Seq	RD <= RS == RT ? 1 : 0	
Deslocamento		
ShiftR [RD, RS]	RD <= Desl(RS) p/ a direita uma vez	R
ShiftL [RD, RS]	RD <= Desl(RS) p/ a esquerda uma vez	R
Jump instructions		
Jump [ENDEREÇO]	PC <= ENDEREÇO	J
JR [RS]	PC <= RS	J
JAL [RS, ENDEREÇO]	RS <= PC(ENDEREÇO)	J
Branch instructions		
BEQ [RD, RS, ENDEREÇO]	IF(RD == RS), PC <= ENDEREÇO	I
BNE [RD, RS, ENDEREÇO]	IF(RD != RS), PC <= ENDEREÇO	I
Transferência		
Move [RD, RS]	RD <= RS	R
Load in register		
Load [RD, ENDEREÇO]	RD <= Ram_dados [ENDEREÇO]	I
Loadl [RD, IM14]	RD <= IM14	I
Store in memory		
Store [RD, ENDEREÇO]	Ram_dados [ENDEREÇO] <= RD	I
Input and output		
In [RD, INPUT]	RD <= INPUT	R
Out [RD, OUTPUT]	OUTPUT <= RD	R
Controle		
Nop []	Nenhuma operação	ESPECIAIS
Hlt []	Parar operação	ESPECIAIS

OPCODE	INSTRUÇÃO	OPCODE	INSTRUÇÃO
000000	Add	001110	Jump
000001	Addl	001111	JR
000010	Sub	010000	JAL
000011	Subl	010001	BEQ

000100	Div	010010	BNE
000101	Divl	010011	Move
000110	Mult	010100	Load
000111	Multl	010101	Loadl
001000	Or	010110	Store
001001	And	010111	In
001010	Xor	011000	Out
001011	Not	011001	Nop
001100	ShiftR	011010	Hlt
001101	ShiftL		

A memória é organizada de forma estática: o endereço de cada variável é fixado em tempo de compilação, uma decisão de projeto imposta pela implementação da instrução Load, que opera com endereços absolutos.

3. O Compilador: Fase de Análise

A fase de análise é o primeiro estágio da compilação, onde o código-fonte é decomposto, validado e transformado em uma representação intermediária estruturada. Este processo é dividido em três subetapas canônicas: análise léxica, sintática e semântica.

3.1. Análise Léxica: A Formação de Tokens

O analisador léxico, implementado com a ferramenta

Flex, varre o código-fonte caractere por caractere e agrupa sequências de caracteres em unidades lexicais chamadas **tokens** (ex: palavras-chave, identificadores, operadores). Expressões regulares são usadas para definir os padrões de cada token válido na linguagem C-.

3.2. Análise Sintática e a Geração da Árvore Sintática Abstrata (AST)

A análise sintática, ou *parsing*, valida a estrutura do programa. Utilizando a ferramenta **Bison**, este processo verifica se a sequência de tokens fornecida pelo analisador léxico conforma-se às regras de uma gramática livre de contexto, que define a sintaxe da linguagem C-. O objetivo principal desta fase, contudo, vai além da simples validação: ela constrói a **Árvore Sintática Abstrata (AST)**, uma representação de dados fundamental para as etapas posteriores.

A construção da AST ocorre de maneira incremental. Para cada regra gramatical definida no arquivo `cminus.y`, uma "ação semântica" em código C é associada. Quando o analisador consegue "reduzir" uma sequência de tokens a uma regra gramatical, a ação correspondente é executada. Essas ações invocam funções auxiliares, como `newStmtNode` (para nós de comando, como `if` ou `while`) e `newExpNode` (para nós de

expressão, como + ou >), que alocam dinamicamente a memória para um novo nó da árvore.

Cada nó recém-criado é então conectado à estrutura da árvore. Por exemplo, em uma expressão $x + y$, o analisador primeiro criaria nós para x e y . Ao reconhecer o operador $+$, ele criaria um nó de expressão para a adição e apontaria os filhos deste nó para os nós de x e y . Estruturas mais complexas, como uma lista de declarações ou comandos, são construídas encadeando os nós horizontalmente através de ponteiros sibling (irmão), formando uma lista ligada de nós no mesmo nível hierárquico. Ao final do processo, a AST representa toda a estrutura lógica do programa, pronta para ser consumida pela análise semântica e, posteriormente, pela geração de código.

3.3. Análise Semântica e a Tabela de Símbolos

A análise semântica verifica o significado do código, garantindo que ele faça sentido do ponto de vista lógico. Para isso, ela percorre a AST e utiliza uma estrutura de dados crucial: a **Tabela de Símbolos**.

A Tabela de Símbolos armazena informações sobre todos os identificadores (nomes de variáveis e funções) do programa. Cada entrada na tabela contém atributos como o tipo do símbolo (inteiro, vetor), seu escopo (local ou global) e seu endereço de memória alocado. Durante a travessia da AST, o analisador semântico popula a tabela com as declarações encontradas e a consulta para realizar verificações essenciais, como:

- **Checagem de Tipos:** Garante que os operandos de uma expressão sejam de tipos compatíveis (ex: não se pode somar um número com um vetor).
- **Verificação de Escopo:** Assegura que variáveis e funções sejam declaradas antes de serem usadas e que sejam acessadas apenas dentro do escopo em que são visíveis.
- **Gerenciamento de memória:** Assegura que os endereços sejam corretamente alocados para cada variável

4. O Compilador: Fase de Síntese

Após a validação do código, a fase de síntese traduz a AST em código executável para o processador-alvo.

4.1. Geração de Código Intermediário: As Quádruplas

Após a validação semântica, a fase de síntese inicia a tradução da AST para um formato mais linear e próximo da máquina, conhecido como **código intermediário**. Este compilador utiliza **quádruplas**, uma forma de código de três endereços onde cada instrução representa uma operação atômica.

A geração é realizada por uma função recursiva, *cgen*, que percorre a AST em pós-ordem. Para cada nó visitado, a função determina o tipo de construção (expressão, atribuição, desvio) e emite as quádruplas apropriadas:

- **Expressões:** Para um nó de expressão como +, a função `cgen` é chamada recursivamente para seus filhos (os operandos). Cada chamada retorna um registrador temporário (R1, R2, etc.) que armazena o resultado daquele subárvore. Em seguida, uma única quádrupla, como (ADD, R1, R2, R3), é gerada para realizar a soma, armazenando o resultado em um novo temporário, R3.
- **Controle de Fluxo:** Estruturas como `if` e `while` são traduzidas usando quádruplas de salto. Para um comando `if`, por exemplo, o compilador gera uma quádrupla de salto condicional, como `JUMP_FALSE(salte se for falso)`, que desvia a execução para um rótulo (LABEL) posicionado após o bloco `then`. Um salto incondicional (JUMP) é inserido ao final do bloco `then` para pular o bloco `else`. Rótulos únicos são gerados sob demanda pela função `newLabel`.
- **Chamada de Funções:** Chamadas de função são decompostas em múltiplas quádruplas: `PARAM` para cada argumento passado, `CALL` para realizar o salto e `RETURN` para o retorno.

Este processo converte a estrutura hierárquica e complexa da AST em uma lista simples e sequencial de operações atômicas, que é a base para a geração do código de máquina final.

4.2. Geração de Código Assembly

A geração do código Assembly é o processo de traduzir a lista de quádruplas do código intermediário para a linguagem de montagem específica do processador-alvo. Essa etapa é notavelmente direta, pois a simplicidade das quádruplas e a filosofia RISC do processador se alinham bem.

Uma função, `generate_assembly`, itera sobre a lista de quádruplas. Dentro dela, uma estrutura `switch` mapeia cada tipo de operação da quádrupla (`OpKind`) para uma ou mais instruções Assembly:

- **Operações Aritméticas:** Quádruplas como (ADD, R1, R2, R3) são traduzidas diretamente para a instrução Assembly correspondente `add r1, r2, r3`.
- **Acesso à Memória:** (LOAD, end, R1) e (STORE, end, t1) são convertidas para `load r1, end` e `store r1, end`, respectivamente, utilizando os endereços estáticos calculados na análise semântica.
- **Desvios:** A quádrupla (JUMP_FALSE, r1, L1) é traduzida para `beq r1, r63, L1`. Esta instrução desvia para o rótulo L1 se o valor no registrador temporário r1 for igual a zero (já que r63 é o registrador que contém permanentemente o valor zero). A quádrupla (JUMP, L1) é convertida diretamente para `jump L1`.
- **Chamadas de Função:** A convenção de chamada é implementada aqui. Quádruplas `PARAM` geram instruções `move` para colocar os argumentos nos registradores `a0`, `a1`, etc. A quádrupla `CALL` é traduzida para `jal` (Jump and Link), que salva o endereço de retorno em `ra` e salta para a função.

4.3. Geração do Código Executável (Binário)

A etapa final do processo de compilação é a conversão do código em linguagem Assembly, legível por humanos, para um código de máquina binário, que pode ser diretamente executado pelo processador. Essa tarefa é realizada por um componente chamado **Montador** (Assembler). O montador implementado (binary.c) utiliza uma abordagem clássica e robusta conhecida como **montagem de dois passos** (two-pass assembly).

Essa abordagem é necessária para resolver um problema fundamental na tradução de código: a **referência futura** (forward reference). Isso ocorre quando uma instrução faz referência a um rótulo (label) que só é definido mais adiante no código (ex: `j Loop` antes da linha `Loop:`). Sem saber o endereço de memória de `Loop`, é impossível gerar a instrução `j` corretamente. O montador de dois passos soluciona isso percorrendo o arquivo duas vezes.

Passo 1: Mapeamento de Rótulos e Endereços

O objetivo da primeira passagem é unicamente construir um mapa completo de todos os rótulos declarados no arquivo Assembly e seus respectivos endereços de memória. Nenhum código binário é gerado nesta fase.

O processo de percurso funciona da seguinte maneira:

1. **Inicialização:** O montador inicializa dois contadores de endereço: um para a seção de dados (`.data`) e outro para a seção de texto/código (`.text`). Ele também inicializa uma tabela global de rótulos (`g_label_table`).
2. **Leitura do Arquivo:** O arquivo de entrada `.s` é lido linha por linha, do início ao fim.
3. **Identificação da Seção:** O montador verifica se a linha indica uma mudança de seção (para `.data` ou `.text`) para saber qual contador de endereço deve usar.
4. **Detecção de Rótulos:** Cada linha é analisada em busca do caractere `:`, que sinaliza a definição de um rótulo.
5. **Armazenamento na Tabela:** Ao encontrar um rótulo (ex: `main:`), seu nome ("main") é extraído e armazenado na `g_label_table` junto com o valor atual do contador de endereço da seção correspondente.
6. **Incremento de Endereço:** Após processar o rótulo, o montador verifica se a mesma linha contém uma instrução ou uma diretiva de dados (como `.word`). Se houver, ele incrementa o contador de endereço apropriado (`text_address_counter` para uma instrução, `data_address_counter` para um `.word`), reservando espaço na memória para aquele item.

Ao final desta passagem, a `g_label_table` contém o endereço exato de cada rótulo no programa, como `main`, `Loop`, `var1`, etc.

Passo 2: Geração do Código Binário

Com o mapa de rótulos completo, a segunda passagem percorre o arquivo novamente para realizar a tradução final.

1. **Reinicialização:** O montador reposiciona a leitura para o início do arquivo `.s` e abre o arquivo de saída `.bin` em modo de escrita binária (`"wb"`).
2. **Percorso e Análise:** O arquivo é lido linha por linha uma segunda vez. O montador novamente rastreia se está na seção `.data` ou `.text`.
 - **Na Seção `.data`:** Se uma diretiva como `.word` é encontrada, o valor numérico que a segue é convertido para uma palavra de 32 bits e escrito diretamente no arquivo de saída binário usando a função `write_true_binary_word`.
 - **Na Seção `.text`:** Este é o núcleo da tradução. Para cada linha contendo uma instrução:
 - a. **Análise da Linha (Parsing):** A função `parse_line` isola o mnemônico da instrução (ex: `add`, `beq`, `load`) e seus argumentos (`$t0`, `$t1`, `var1`, etc.).
 - b. **Identificação da Instrução:** O mnemônico é procurado na tabela de opcodes (`g_opcode_table`) para obter seu código de operação binário (`opcode`) e, mais importante, seu **formato** (ex: `FORMAT_1`, `FORMAT_7`). O formato define a quantidade e o tipo dos operandos esperados.
 - c. **Montagem Específica ao Formato:** A função `assemble_instruction` utiliza uma estrutura `switch` baseada no formato da instrução para processar os argumentos corretamente.
 - d. **Resolução de Símbolos:** Para cada argumento, o montador realiza uma consulta:
 - Se for um registrador (ex: `$t0`), a função `get_register_number` o converte para seu valor numérico (ex: 8).
 - Se for um rótulo (ex: `var1` em uma instrução `load` ou `Loop` em uma `j`), a função `get_label_address` consulta a `g_label_table` (preenchida no Passo 1) para obter seu endereço de memória.
 - Se for um valor numérico imediato, ele é convertido diretamente.
 - e. **Construção da Palavra Binária:** Com todos os componentes traduzidos para seus valores numéricos (`opcode`, registradores, endereços), a função `build_instruction` os combina em uma única palavra de 32 bits, utilizando operações de deslocamento de bits (`<<`) e OU (`|`) para posicionar cada campo no local correto dentro da instrução.
 - f. **Escrita no Arquivo:** A palavra final de 32 bits é escrita no arquivo de saída através da função `write_true_binary_word`.

Em suma, o montador funciona através de uma estratégia de "dividir para conquistar": a primeira passagem coleta todas as informações de endereçamento simbólico, e a segunda utiliza esse mapa para traduzir sistematicamente cada instrução, garantindo que todas as referências, futuras ou não, sejam resolvidas corretamente.

Análise da Integração: A Jornada de uma Instrução

Para compreender como todas as fases do compilador se integram, podemos acompanhar a jornada de uma única e simples linha de código C- através de todo o processo. Considere a instrução: `y = x + 10;`

1. **Análise Léxica:** O código fonte é lido e decomposto em uma sequência de tokens: IDENTIFICADOR(y), ATRIBUICAO(=), IDENTIFICADOR(x), OPERADOR(+), NUMERO(10), PONTO_E_VIRGULA(;).
2. **Análise Sintática:** O analisador sintático (parser) consome essa sequência de tokens. Usando as regras da gramática, ele reconhece a estrutura de uma atribuição e constrói um pequeno galho na Árvore Sintática Abstrata (AST). O nó raiz seria um nó de = (atribuição), com dois filhos: à esquerda, um nó para a variável y, e à direita, um nó para a expressão +. Este nó de +, por sua vez, teria dois filhos: um para a variável x e outro para a constante 10.
3. **Análise Semântica:** A árvore é percorrida. O analisador consulta a Tabela de Símbolos para encontrar as informações de x e y, como seus tipos (ex: Integer) e seus endereços de memória estáticos (ex: x no endereço 0, y no endereço 1). Ele verifica se os tipos são compatíveis para a soma e a atribuição.
4. **Geração de Código Intermediário:** A função cgen percorre a sub-árvore da AST e a traduz em quádruplas lineares:
 - (LOAD, r1, 0) – Carrega o valor de x (do endereço 0) em um registrador temporário t1.
 - (LOADI, r2, 10) – Atribui a constante 10 a um temporário t2 (LOADI é uma operação de atribuição).
 - (ADD, r1, r2, r3) – Soma r1 e r2, armazenando o resultado em um novo temporário r3.
 - (STORE, r3, 1) – Armazena o resultado de r3 na localização de memória de y (endereço 1).
5. **Geração de Código Assembly:** Cada quádrupla é mapeada para uma instrução MIPS:
 - load r1, x
 - loadI r2, 10
 - add r1, r2, r3
 - store r3, y
6. **Geração do Código Executável (Binário):** O montador de dois passos converte cada linha de Assembly em uma palavra binária de 32 bits. Para add r1, r2, r3, por exemplo, ele combinaria o opcode de add, os números dos registradores r1, r2, r3 e outros campos, resultando em uma única instrução binária.

Essa jornada demonstra um fluxo contínuo de transformação, onde cada fase refina e traduz a representação do programa, aproximando-a progressivamente da linguagem que o hardware pode executar diretamente, ilustrando a notável integração de teoria e prática na construção de um compilador.

4.5. Gerenciamento de Memória: Alocação Estática

O gerenciamento de memória deste compilador é baseado em um modelo de **alocação puramente estática**, o que significa que o endereço de cada variável, seja ela global ou local, é determinado em tempo de compilação. Durante a fase de análise semântica, à medida que o compilador processa as declarações, um contador de deslocamento de

memória atribui um endereço fixo e único para cada variável dentro de seu escopo. Esse endereço é então armazenado na Tabela de Símbolos junto com as outras informações do identificador.

Essa abordagem difere do modelo padrão utilizado pela maioria dos compiladores modernos, que empregam uma pilha de execução dinâmica gerenciada por ponteiros como o ponteiro de pilha (*stack pointer*, sp) e o ponteiro de quadro (*frame pointer*, fp). A escolha pelo modelo estático foi uma consequência direta e uma adaptação necessária às restrições do hardware-alvo. A arquitetura do processador desenvolvido possui uma instrução load projetada para operar com um endereço de memória absoluto, no formato Load [RD, ENDEREÇO].

Para implementar um gerenciamento de pilha dinâmico, seria necessário que o processador suportasse instruções capazes de acessar a memória a partir de um deslocamento (offset) relativo a um registrador base, como load rd, offset(rs). Como o hardware não possui tal instrução, a manipulação de um ponteiro de pilha tornou-se inviável. Portanto, o design do compilador foi direcionado para a estratégia de calcular todos os endereços de forma estática, utilizando esses endereços fixos armazenados na tabela de símbolos para emitir as instruções load e store. Embora menos flexível que uma pilha dinâmica (especialmente para recursão profunda), este método é eficiente e perfeitamente adequado às limitações da linguagem C- e do processador-alvo.

5. Exemplos de Compilação

A seguir, são apresentados três exemplos que demonstram o funcionamento completo do compilador.

5.0. Exemplo 0: Contador

Esse programa lê um valor e conta de 0 até esse valor.

Código Fonte em C-:

```
void main(void)
{
    int x; int y;
    x = 0;
    y = input();
    while (x < y)
    {
        output(x);
        x = x + 1;
    }
}
```

- **Código Intermediário (Quádruplas):**

0: (FUNC, main, - , -)

```

1: (LOADI, 0, , R0)
2: (STORE, x, , R0)
3: (CALL, input, 0, R1)
4: (STORE, y, , R1)
5: (LABEL, _, _, L0)
6: (LOAD, x, -, R2)
7: (LOAD, y, -, R3)
8: (LT, R2, R3, R4)
9: (JUMP_FALSE, R4, -, L1)
10: (LOAD, x, , R5)
11: (PARAM, R5, , )
12: (CALL, output, 1, R6)
13: (LOAD, x, -, R6)
14: (LOADI, 1, , R7)
15: (ADD, R6, R7, R8)
16: (STORE, x, , R8)
17: (JUMP, _, _, L0)
18: (LABEL, _, _, L1)
19: (END FUNCTION, main, -, -)

```

- **Código Assembly (MIPS):**

```
.data
```

```
x:      .word 0
```

```
y:      .word 0
```

```
.text
```

```
.globl main
```

main:

loadi r0, 0

store r0, x

in r1

store r1, y

L0:

load r2, x

load r3, y

slt r4, r2, r3

beq r4, r63, L1

load r5, x

out

load r6, x

loadi r7, 1

add r8, r6, r7

store r8, x

jump L0

L1:

jump exit_program

exit_program:

hlt

- **Código Executável (Binário):**

01010100000000000000000000000000

```

01011000000000000000000000000000
01011100000100000000000000000000
01011000000100000000000000000001
01010000001000000000000000000000
01010000001100000000000000000001
01110000001000001100010000000000
01000111111100010000000000001000
01010000010100000000000000000000
11111111111100000000000000000000
01010000011000000000000000000000
01010100011100000000000000000001
00000000011000011100100000000000
01011000100000000000000000000000
0011100000000000000000000000100
00111000000000000000000000001000
01101000000000000000000000000000

```

5.1. Exemplo 1: Maior de Dois Números

Este programa lê dois inteiros e imprime o maior deles.

Código Fonte em C-:

```

void main(void)
{
    int x; int y;
    x = input();
    y = input();
    if (x>y)
        output(x);
    else
        output(y);
}

```

- **Código Intermediário (Quádruplas):**

```

0: (FUNC, main, -, -)
1: (CALL, input, 0, R0)
2: (STORE, x, , R0)
3: (CALL, input, 0, R1)
4: (STORE, y, , R1)
5: (LOAD, x, -, R2)
6: (LOAD, y, -, R3)
7: (GT, R2, R3, R4)
8: (JUMP_FALSE, R4, _, L0)
9: (LOAD, x, , R5)
10: (PARAM, R5, , )
11: (CALL, output, 1, R6)
12: (JUMP, _, _, L1)
13: (LABEL, _, _, L0)
14: (LOAD, y, , R6)
15: (PARAM, R6, , )
16: (CALL, output, 1, R7)
17: (LABEL, _, _, L1)
18: (END FUNCTION, main, -, -)

```

- **Código Assembly (MIPS):**

```
.data
```

```
x:      .word 0
```

```
y:      .word 0
```

```
.text
```

```
.globl main
```

main:

in r0

store r0, x

in r1

store r1, y

load r2, x

load r3, y

slt r4, r3, r2

beq r4, r63, L0

load r5, x

out

jump L1

L0:

load r6, y

out

L1:

jump exit_program

exit_program:

hlt

- **Código Executável (Binário):**

01011100000000000000000000000000

01011000000000000000000000000000

01011100000100000000000000000000


```
0101100000010000000000000000000001
0101000000100000000000000000000000
0101000000110000000000000000000001
0111000000110000100001000000000000
0100011111110001000000000000000100
0101000001010000000000000000000000
1111111111110000000000000000000000
00111000000000000000000000000001101
0101000001100000000000000000000001
1111111111110000000000000000000000
00111000000000000000000000000001110
0110100000000000000000000000000000
```

5.2. Exemplo 2: Cálculo de Fatorial

Este programa calcula o fatorial de um número usando um laço de repetição `while`.

Código Fonte em C-:

```
void main(void)
{
    int fat;
    int n;
    fat = 1;

    n = input();
    output(n);

    while(n > 1){
        fat = fat * n;
        output(fat);
        n = n - 1;
    }

    output(fat);
}
```

- **Código Intermediário (Quádruplas):**

0: (FUNC, main, -, -)

1: (LOADI, 1, , R0)

2: (STORE, fat, , R0)

3: (CALL, input, 0, R1)

4: (STORE, n, , R1)

5: (LOAD, n, , R2)

6: (PARAM, R2, ,)

7: (CALL, output, 1, R3)

8: (LABEL, _, _, L0)

9: (LOAD, n, -, R3)

10: (LOADI, 1, , R4)

11: (GT, R3, R4, R5)

12: (JUMP_FALSE, R5, -, L1)

13: (LOAD, fat, -, R6)

14: (LOAD, n, -, R7)

15: (MUL, R6, R7, R8)

16: (STORE, fat, , R8)

17: (LOAD, fat, , R9)

18: (PARAM, R9, ,)

19: (CALL, output, 1, R10)

20: (LOAD, n, -, R10)

21: (LOADI, 1, , R11)

22: (SUB, R10, R11, R12)

23: (STORE, n, , R12)

24: (JUMP, _, _, L0)

25: (LABEL, _, _, L1)

26: (LOAD, fat, , R13)

27: (PARAM, R13, ,)

28: (CALL, output, 1, R14)

29: (END FUNCTION, main, - , -)

- **Código Assembly (MIPS):**

.data

fat: .word 0

n: .word 0

.text

.globl main

main:

loadi r0, 1

store r0, fat

in r1

store r1, n

load r2, n

out

L0:

load r3, n

loadi r4, 1

slt r5, r4, r3

beq r5, r63, L1

load r6, fat

```
load r7, n
```

```
mult r8, r6, r7
```

```
store r8, fat
```

```
load r9, fat
```

out

```
load r10, n
```

```
loadi r11, 1
```

```
sub r12, r10, r11
```

```
store r12, n
```

jump L0

L1:

```
load r13, fat
```

out

jump exit_program

exit_program:

hlt

- **Código Executável (Binário):**

010101000000000000000000000000000001

01011000000000000000000000000000000000

01011100000100000000000000000000

010110000001000000000000000000000001

010100000010000000000000000000000001

1111111111000000000000000000

01010000001100000000000000000001

```

010101000100000000000000000000000001
01110000010000001100010100000000
01000111111100010100000000001100
01010000011000000000000000000000
0101000001110000000000000000000001
00011000011000011100100000000000
01011000100000000000000000000000
01010000100100000000000000000000
11111111111100000000000000000000
0101000010100000000000000000000001
0101010010110000000000000000000001
00001000101000101100110000000000
0101100011000000000000000000000001
001110000000000000000000000000110
01010000110100000000000000000000
11111111111100000000000000000000
00111000000000000000000000000011000
01101000000000000000000000000000

```

5.3. Exemplo 3: Máximo Divisor Comum (Recursivo)

Este exemplo implementa o algoritmo de Euclides de forma recursiva para calcular o MDC. Notavelmente, o compilador aplicou a otimização de chamada de cauda (Tail-Call Optimization), transformando a recursão em iteração para economizar pilha.

Código Fonte em C-:

```

int gcd (int u, int v)
{
    if (v == 0) return u ;
    else return gcd(v,u-u/v*v);
    /* u-u/v*v == u mod v */
}

```

```

void main(void)
{
    int x; int y;
    x = input(); y = input();
    output(gcd(x,y));
}

```

- **Código Intermediário (Quádruplas):**

```

0: (FUNC, gcd, -, -)

1: (LOAD, v, -, R0)

2: (LOADI, 0, , R1)

3: (EQ, R0, R1, R2)

4: (JUMP_FALSE, R2, _, L0)

5: (LOAD, u, -, R4)

6: (RETURN, u, _, R4)

7: (JUMP, _, _, L1)

8: (LABEL, _, _, L0)

9: (LOAD, v, , R5)

10: (PARAM, R5, , )

11: (LOAD, u, -, R6)

12: (LOAD, u, -, R7)

13: (LOAD, v, -, R8)

14: (DIV, R7, R8, R9)

15: (LOAD, v, -, R10)

16: (MUL, R9, R10, R11)

17: (SUB, R6, R11, R12)

18: (PARAM, R12, , )

19: (CALL, gcd, 2, R13)

20: (RETURN, gcd, _, R13)

21: (LABEL, _, _, L1)

```

22: (END FUNCTION, gcd, -, -)

23: (FUNC, main, -, -)

24: (CALL, input, 0, R14)

25: (STORE, x, , R14)

26: (CALL, input, 0, R15)

27: (STORE, y, , R15)

28: (LOAD, x, , R16)

29: (PARAM, R16, ,)

30: (LOAD, y, , R17)

31: (PARAM, R17, ,)

32: (CALL, gcd, 2, R18)

33: (PARAM, R18, ,)

34: (CALL, output, 1, R19)

35: (END FUNCTION, main, -, -)

- **Código Assembly (MIPS):**

.data

x: .word 0

y: .word 0

.text

.globl main

gcd:

 load r0, v

 loadi r1, 0

 seq r2, r0, r1

beq r2, r63, L0

load r4, u

move r2, r4

jump L1

L0:

load r5, v

load r6, u

load r7, u

load r8, v

div r9, r7, r8

load r10, v

mult r11, r9, r10

sub r12, r6, r11

move r4, r5

move r5, r12

jal gcd

move r13, r2

move r2, r13

L1:

jr r62

main:

in r14

store r14, x

in r15

store r15, y


```
load r16, x
load r17, y
move r4, r16
move r5, r17
jal gcd
move r18, r2
out r16
jump exit_program
```

exit_program:

hlt

- **Código Executável (Binário):**

```
01010000000000000000000000000000
01010100000100000000000000000000
01110100000000000100001000000000
01000111111100001000000000000100
01010000010000000000000000000000
01001100010000001000000000000000
001110000000000000000000000010100
01010000010100000000000000000000
01010000011000000000000000000000
01010000011100000000000000000000
01010000100000000000000000000000
00010000011100100000100100000000
01010000101000000000000000000000
```

00011000100100101000101100000000
00001000011000101100110000000000
01001100010100010000000000000000
01001100110000010100000000000000
01000001111100000000000000000000
01001100001000110100000000000000
01001100110100001000000000000000
00111111110000000000000000000000
01011100111000000000000000000000
01011000111000000000000000000000
01011100111100000000000000000000
01011000111100000000000000000001
01010001000000000000000000000000
01010001000100000000000000000001
01001101000000010000000000000000
01001101000100010100000000000000
01000001111100000000000000000000
01001100001001001000000000000000
01100001000000000000000000000000
0011100000000000000000000000100001
01101000000000000000000000000000

6. Conclusão

A construção de um compilador completo, desde a análise de um arquivo de texto até a geração de um código binário executável, representou um dos desafios mais complexos e recompensadores da graduação. Este projeto foi uma oportunidade ímpar de solidificar a compreensão da interação fundamental entre software e hardware, desmistificando as camadas de abstração que separam um comando de alto nível da sua execução física no processador.

Contrariando a expectativa de que as etapas finais seriam as mais árduas, a principal dificuldade técnica residiu na **geração do código intermediário a partir da Árvore Sintática Abstrata (AST)**. Esta fase se revelou o verdadeiro coração intelectual do compilador. O desafio não estava apenas em percorrer uma estrutura de dados, mas em traduzir a lógica hierárquica e abstrata da árvore em uma representação linear e sequencial de quádruplas. A conversão de estruturas de controle de fluxo, como `while` e `if-else`, foi particularmente complexa, pois exigiu a invenção de um sistema de gerenciamento de rótulos (`labels`) e a inserção precisa de desvios condicionais (`OP_IF_F`) e incondicionais (`OP_GOTO`) para recriar a semântica do programa original de forma linear. A gestão de registradores temporários para expressões aninhadas e a decomposição de chamadas de função em múltiplas operações atômicas (`PARAM`, `CALL`) demandaram um raciocínio algorítmico cuidadoso e uma atenção minuciosa aos detalhes.

Uma vez que essa tradução complexa para o código intermediário foi bem-sucedida, a etapa seguinte — a geração do código Assembly — tornou-se uma tarefa consideravelmente mais direta. O trabalho árduo já havia sido feito; a lógica de controle já estava explicitada em saltos e rótulos, tornando a conversão para as instruções MIPS uma correspondência quase direta. Ainda assim, foi preciso adaptar o compilador a um hardware pré-existente, com suas próprias restrições. A decisão de implementar um gerenciamento de memória puramente estático, por exemplo, foi uma consequência direta da arquitetura da instrução `load` do processador, demonstrando na prática como a arquitetura do hardware impõe condições reais ao design do software.

No entanto, o maior aprendizado foi a lição de perseverança. Superar os inúmeros desafios, depurar problemas complexos e, finalmente, ver o código-fonte ser transformado em um programa executável funcional no processador desenvolvido foi uma experiência imensamente recompensadora. O processo de depuração frequentemente nos forçava a navegar por todas as camadas: de um erro na saída final, subindo pelo código binário e Assembly, para então descobrir uma falha na lógica de geração das quádruplas. A maior satisfação, portanto, não foi apenas entregar o projeto, mas ter dominado a etapa mais abstrata e desafiadora de todo o processo, compreendendo em profundidade como transformar a estrutura elegante de uma árvore sintática na sequência rigorosa de passos que, ao final, comanda o hardware. Esse foi, sem dúvida, o aprendizado mais valioso de todo o processo.

7. Referências

[1] LOUDEN, K. C. *Compiler Construction: Principles and Practice*. PWS Publishing, 1997.